LitCVit: A Lightweight Self-Supervised Contrastive Vision Transformer for Encrypted Malicious Traffic Detection

Mehr Un Nisa, Adnan Noor Mian, *Senior Member, IEEE*, and Mubashir Husain Rehmani, *Senior Member, IEEE*

***Abstract*—Malicious traffic detection often requires large, labeled datasets, which are challenging due to privacy concerns, labeling costs, and evolving threat patterns. Although recent self-supervised pretraining methods address this issue, they rely on complex transformer-based architectures that are computationally expensive and have high inference times, making them unsuitable for real-time use. In addition, most existing approaches process packets or flows independently, and often rely on per-packet dataset splits that introduce implicit flow-level data leakage, thereby limiting their ability to capture meaningful semantic and behavioral relationships across flows for detecting stealthy encrypted threats. To address these issues, we propose LitCVit, a lightweight self-supervised contrastive Vision Transformer-based framework that captures cross-flow semantic and behavioral patterns to generate robust latent representations of encrypted traffic. Without relying on decryption or manually engineered features, our method enables efficient detection of encrypted malicious flows with low inference time. Extensive evaluations on benchmark datasets demonstrate that the proposed framework achieves an average detection accuracy of 98.10% and F1-score of 98.08%. Compared to the best state-of-the-art model, LitCVit achieves an average improvement of 2.49% in F1-score, 2.12% in precision, and 2.50% in recall, highlighting its superior detection capability in encrypted traffic scenarios. Additionally, LitCVit achieves an 8.7× reduction in inference time compared to the best existing self-supervised approach, making it highly suitable for deployment on resource-constrained devices.**

***Index Terms*—Encrypted malicious traffic, self-supervised malicious traffic detection, pre-training models, contrastive learning, malware or attack classification.**

I. Introduction

THE rapid proliferation of encrypted traffic has not only enhanced user security and privacy but also created new opportunities for adversaries to conceal malicious activities within encrypted payloads, facilitating sophisticated attacks such as command-and-control (C2). In parallel, the rise of cyber-physical systems and the Internet of Things (IoT) has further expanded the threat landscape, as cybercriminals increasingly exploit these encrypted channels to launch persistent and evasive attacks, effectively bypassing conventional defenses [1]. Additionally, advancements in encryption protocols, such as TLS 1.3 and DNS over HTTPS, have reduced the availability of previously accessible metadata [2]. These trends emphasize the urgent need for robust and efficient detection techniques to identify malicious patterns within encrypted network traffic.

Artificial intelligence (AI)-based techniques have emerged as an effective alternative to rule-based and signature-based methods for detecting encrypted malicious traffic [3]. Most of these methods rely on manually engineered features extracted from protocol headers, flow, or handshake metadata to identify malicious patterns. However, these methods provide limited visibility into fine-grained encrypted network traffic, leading to higher false alarm rates (FAR), challenges in high-speed traffic processing, and poor generalization across diverse network conditions [4].

End-to-end deep learning methods address the limitations of manual feature engineering by automatically learning relevant features from raw data. However, they require large volumes of labeled data for effective training to achieve satisfactory performance. Acquiring and labeling such data across diverse network environments is a time-consuming and resource-intensive process, often introducing significant security and privacy risks [5]. Self-supervised approaches have emerged to address these challenges by leveraging large volumes of unlabeled data for pretraining. These methods reduce the reliance on labeled datasets and improve generalization across diverse network environments. However, they have complex transformer-based architectures [6], [7], [8], [9] which are unsuitable for real-time intrusion detection due to high computational and latency requirements. Additionally, most techniques process packets independently and adopt per-packet data splits [6], [10], [11], which introduce implicit flow-level data leakage [9] and fail to capture meaningful semantic and behavioral relationships across flows necessary for detecting sophisticated threats.

In this paper, we propose LitCVit, a lightweight contrastive Vision Transformer-based framework, for encrypted malicious traffic detection. Our core idea is to efficiently capture features from raw encrypted traffic at both fine and coarse grained levels through a hierarchy of lightweight encoder blocks.

Received 21 August 2025; revised 10 March 2026; accepted 6 April 2026. Date of publication 13 April 2026; date of current version 21 April 2026. The associate editor coordinating the review of this article and approving it for publication was Dr. Meng Li. *(Corresponding author: Adnan Noor Mian.)*

Mehr un Nisa and Adnan Noor Mian are with the Cyber Intelligence Laboratory, Department of Computer Science, Information Technology University, Lahore 54600, Pakistan (e-mail: mehr.nisa@itu.edu.pk; adnan.noor@itu.edu.pk).

Mubashir Husain Rehmani is with the Department of Computer Science, Munster Technological University, Cork, T12 P928 Ireland (e-mail: mshrehmani@gmail.com).

Digital Object Identifier 10.1109/TIFS.2026.3683528

Moreover, we model semantic and behavioral relationships across flows by clustering semantically similar traffic patterns through contrastive representation learning, aligning flows with similar behaviors in the latent space rather than modeling explicit causal interactions. This enables the model to learn compact and discriminative representations of encrypted traffic flows without requiring large labeled datasets. To enhance computational efficiency, we propose a simple yet effective lightweight architecture that replaces traditional heavy transformer designs, significantly reducing complexity and memory overhead while preserving critical traffic features.

The proposed design significantly reduces trainable parameters (0.658 million) and inference latency (2.32 milliseconds) while preserving strong detection performance. Given its compact design, LitCVit aligns well with the resource constraints of typical edge and IoT devices, as prior studies [12] indicate that models with sub-million parameters, FLOPs in the range of a few million, and inference latencies between $10-500$ ms, are considered practically deployable in such environments. Our main contributions are summarized as follows.

- We propose LitCVit framework, which introduces a lightweight architecture tailored for encrypted malicious traffic detection through two key design strategies:
  - *Firstly*, LitCVit integrates an efficient convolutional patchify stem and three hierarchical windowed factorized attention (wf-Attention) modules, designed to capture fine-grained intra-flow patterns at byte, packet, and flow levels. This architecture significantly reduces the computational overhead of the vanilla Vision Transformer (ViT) architecture while preserving fine-grained spatial representations of flows.
  - *Secondly*, LitCVit employs contrastive learning to capture semantic and behavioral relationships across flows. By clustering ET-flow with similar patterns, the model learns robust and discriminative representations that separate malicious and benign traffic, thereby enhancing detection performance and generalization across diverse threat scenarios.
- We perform a comprehensive computational complexity analysis, demonstrating that the proposed model operates with substantially fewer parameters and reduced memory usage than conventional multi-head self-attention models, making it highly suitable for resource-constrained deployments.
- We validate LitCVit's effectiveness and efficiency across six real-world network traffic datasets, demonstrating its superior performance over state-of-the-art self-supervised methods. The model's lightweight design and low inference latency make it suitable for deployment in resource-constrained environments such as edge and IoT.

The remainder of this paper is organized as follows. Section II reviews related work, Section III introduces the threat model and design goals, Section IV details the architecture of LitCVit, Section V presents the computation complexity analysis, Section VI reports experiments and results, Section VII discusses the findings, Section VIII provides the ablation study, and Section IX concludes the paper.

II. Related Work

We categorize related work into three types: feature-driven, raw-traffic-based, and pre-training-based techniques.

A. Feature-Driven Techniques

Feature-driven methods extract fields from encrypted traffic, including header features [13], flow or connection statistics [14], and side-channel features [15], [16] to infer malicious behavioral signatures. Some techniques analyze Secure Sockets Layer/ Transport Layer Security (SSL/TLS) handshakes for features such as cipher suites [17], JA3/JA3S hashes, and self-signed and expired certificates [18].However, TLS 1.3 handshake encryption and manual feature engineering limit their effectiveness in real-time environments.

B. Raw Bytes-Based Techniques

Raw bytes-based methods use end-to-end deep learning on byte-level traffic via Convolutional Neural Networks (CNNs) [19], [20], hybrid Recurrent Neural Networks (RNNs), Bidirectional Gated Recurrent Units (BiGRUs), and CNNs [21], [22] to capture spatial and temporal features. While effective, they require large labeled datasets, which are difficult to acquire in real time. To overcome this, self-supervised learning techniques have been proposed.

C. Pre-Training-Based Techniques

Pre-training–based methods reduce reliance on labeled data by learning general traffic representations for downstream fine-tuning. Natural language processing (NLP)-based architectures model packet- or flow-level byte sequences as linguistic tokens and apply Large Language Models (LLMs) such as Bidirectional Encoder Representations from Transformers (BERT) [6], [10], [11], [23], a Packet-level End-to-end Attentive Network for encrypted traffic classification (PEAN) [24], a Lite BERT (ALBERT) [25], [26], GPT [27], and T5 [9] to learn representations of encrypted traffic. MIETT [28] further models inter-packet dynamics via multi-instance learning with novel flow-level contrastive pre-training tasks to learn representations of encrypted traffic. Similarly, vision-inspired approaches such as Yet another Traffic Classifier (YaTC) [8], Flow-MAE [29], and NetMamba [30] transform traffic into image-like grids and employ ViT-style architectures. Other methods identify malware traffic using channel-level behavior sequences [7], adversarial pre-training [31], and cross-modal contrastive learning (tFusion [32]).

Despite strong results, recent work reveals that many pre-training–based classifiers suffer from shortcut learning, data leakage, and unrealistic evaluation practices, leading to inflated performance and limited generalization [9], [33]. Moreover, their complex architectures lead to high inference costs. In contrast, Our lightweight ViT-based model addresses these issues through leakage-free evaluation, reduced complexity, and windowed factorized attention with contrastive learning.

III. Threat Model and Design Goals

A. Threat Model

We consider a passive encrypted traffic monitoring scenario where LitCVit operates as a non-intrusive module integrated with a network monitoring device via port mirroring. The adversary communicates over encrypted channels to perform malicious actions such as C2 communication, data exfiltration, ransomware, or denial-of-service (DoS) attacks. We assume the attacker controls the generation of malicious traffic, including payload (prior to encryption), timing behavior, and communication endpoints, but cannot tamper with the monitoring infrastructure. The defender passively observes mirrored traffic, including packet headers and raw encrypted payload bytes, without plaintext access. Labeling is available only during offline fine-tuning, and no online feedback or adaptation occurs during deployment. Although an adaptive attacker may attempt traffic shaping or padding to mimic benign flows, protocol-constrained header fields and encrypted byte patterns limit complete obfuscation, enabling the model to learn robust structural and semantic representations from raw traffic data. The system operates without prior knowledge of attack signatures and learns generalizable flow representations through contrastive self-supervised pretraining across diverse environments and encryption protocols.

B. Design Goals and Problem Definition

We aim to design a lightweight and generalizable framework for encrypted malicious traffic detection. Unlike traditional traffic classification, which identifies applications or users, our goal is to determine whether an ET-flow is benign or malicious by passively analyzing encrypted traffic.

1) Problem Formulation: Let $\mathcal{F} = \{f_1, f_2, \ldots, f_N\}$ denote a set of encrypted traffic flows within a t seconds window, where each flow f_i is uniquely identified by a 5-tuple: $\mathbf{x}_i = \langle \text{srcIP}, \text{srcPort}, \text{dstIP}, \text{dstPort}, \text{protocol} \rangle$ and $f_i = \{p_{i1}, p_{i2}, \ldots, p_{iM}\}$, where each packet $p_{ij} \in \{0,1\}^B$ represents a fixed-length sequence of B bytes. These packet bytes are encoded into a flow matrix $\mathbf{X}_i \in \mathbb{R}^{h \times w}$, which serves as the input representation for the model. The goal is to learn an encoder function $\mathcal{E} : \mathbb{R}^{h \times w} \rightarrow \mathbf{z}_i \in \mathbb{R}^d$ that maps ET-flow images to a low-dimensional latent representation $\mathbf{z}_i$.

During fine-tuning, the pretrained encoder $\mathcal{E}$ is concatenated with a lightweight classifier $\mathcal{C} : \mathbf{z}_i \rightarrow y_i$ and trained using a small labeled dataset $\mathcal{D}_{\text{labeled}} = \{(\mathbf{X}_i, y_i)\}$, where $y_i \in \{0, 1, \ldots, C-1\}$ is the ground truth label. The goal is to accurately detect malicious encrypted flows, while ensuring efficiency, generalization, and low inference latency.

2) Design Goals: To address the scarcity of labeled encrypted malicious traffic, LitCVit aims to learn semantically rich flow-level representations through contrastive self-supervised pretraining, reducing the dependency on manual annotation and rule-based heuristics. The system must be capable of detecting a wide range of threats without relying on protocol-specific features, ensuring robust generalization across diverse encryption protocols and traffic patterns, including 5G and IoT. In addition, the detection pipeline is required to maintain high inference speed and throughput, making it suitable for deployment in high-bandwidth environments.

C. Design Motivation of the Proposed Architecture

The design of LitCVit is motivated by three key limitations of existing approaches. First, although transformer-based architectures provide strong global context modeling, their quadratic attention complexity makes them unsuitable for resource-constrained environments. Second, classical ML methods depend on handcrafted statistical features that offer limited adaptability to modern encrypted protocols such as TLS 1.3. Third, lightweight convolutional models capture local spatial patterns but fail to model long-range behavioral dependencies present in encrypted traffic. To address these limitations, LitCVit adopts a hierarchical Vision Transformer with windowed factorized attention, enabling efficient capture of byte, packet, and flow-level semantics within a compact and lightweight architecture. The comparative and ablation results in Section VIII validate these design choices quantitatively.

IV. Proposed Architecture

In this section, we present the architecture of the proposed framework for encrypted malicious traffic detection, inspired by the ViT [34]. The proposed architecture utilizes a lightweight convolutional patchify stem, which preserves local spatial features. To further enhance efficiency, we incorporate a multi-scale windowed factorized attention mechanism that decomposes attention operations across byte-level, packet-level, and flow-level representations, significantly reducing computational complexity and memory overhead. A contrastive loss term is added to the training objective to produce embeddings that distinguish among various ET-flows during the pre-training phase. The overall architecture consists of three phases: Pre-processing, Pre-training, Fine-tuning & Evaluation phase, as illustrated in Fig. 1. Each of these phases is explained in detail in the following sections.

A. Pre-Processing Phase: Flow-Level Image Construction

To construct a structured and information-preserving representation, we extract ET-flow information directly from the raw Packet Capture (PCAP) file, similar to [8]. Fig. 1 Block A shows the pre-processing phase. We use both unlabeled and labeled PCAP data in preprocessing phase. First, we extract N flows based on IP addresses, port numbers, and transport-layer protocols, port numbers, and the transport layer protocol. Second, each flow is truncated or zero-padded to $M = 5$ packets to ensure fixed-length representation where $p_{1j}, p_{2j}, \ldots, p_{Mj}$ represent the M packets of the j-th flow. Ethernet headers are removed and IP addresses and port numbers are anonymized to ensure user privacy and reduce dataset-specific bias. Each packet is encoded using n bytes, reshaped into a two-dimensional matrix of shape $p \times w$, where p denotes the number of rows and w the number of bytes per row. For a flow consisting of M packets, these matrices are vertically stacked to form a single flow-level matrix:$\mathbf{X}_i \in \mathbb{R}^{(M \cdot p) \times w}$. In our case, we take $p = 8$ and $w = 40$ consisting of $n = 320$ bytes of a packet. Finally, this matrix is saved as an ET-Flow image,

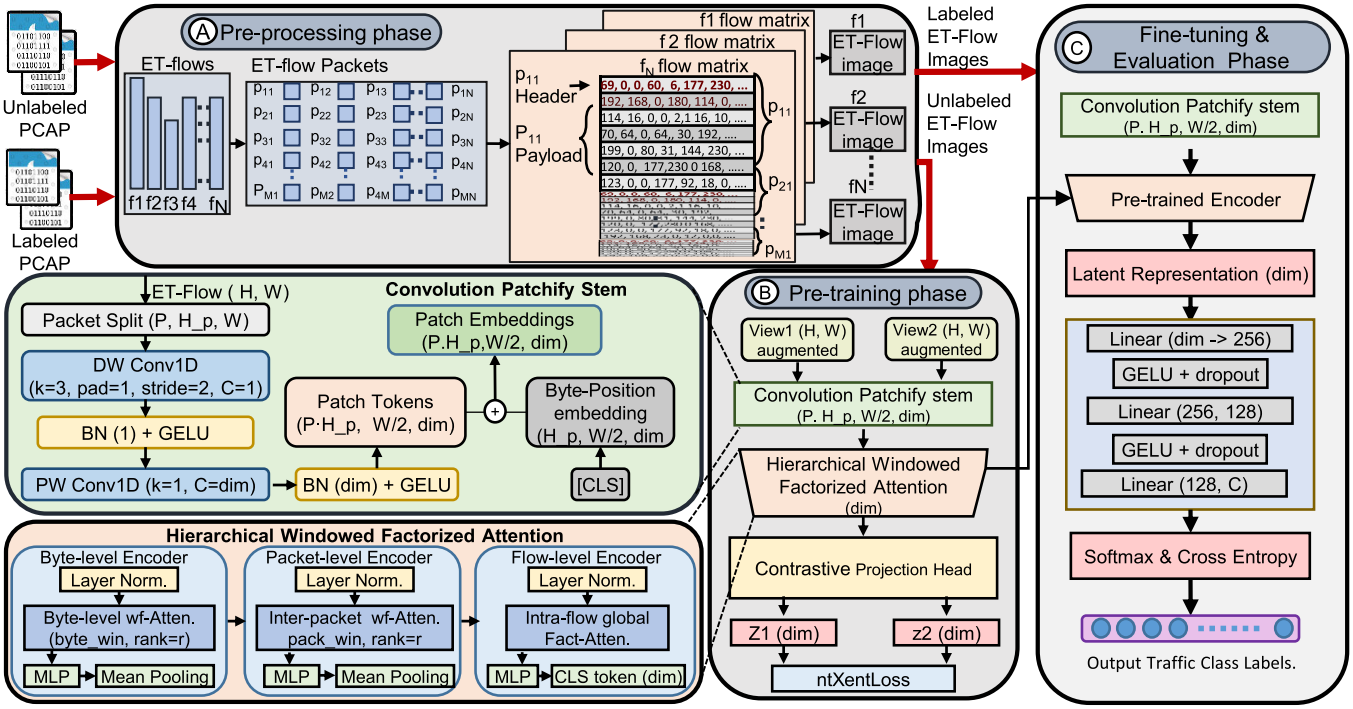

Fig. 1. The proposed framework for encrypted malicious traffic detection.

preserving both the internal byte-level structure of packets and their sequential order. The output of this phase consists of labeled and unlabeled ET-Flow images, to be used for fine-tuning and pre-training, respectively.

B. Pre-Training Phase: Contrastive Representation Learning With ViT

The pre-training phase aims to learn robust and discriminative representations of encrypted traffic flows without requiring labeled data. It consists of three main components: a convolutional patchify stem, hierarchical windowed factorized attention (wf-Attention), and a contrastive embedding head. The overall pre-training process is shown in Fig 1 Block B and described in Algorithm 1. The input to this phase is unlabeled ET-flow images generated during the pre-processing stage. These images are first passed through the convolutional patchify stem, which tokenizes them into patch embeddings. The embeddings are then processed by wf-Attention modules of the transformer, enabling multi-scale feature extraction at byte, packet, and flow levels. Finally, the encoder output is passed through the contrastive learning head to produce robust latent representations for downstream fine-tuning. The details of each component are provided in the following subsections.

1) Convolutional Patchify Stem: Unlike traditional patch embeddings [34] that divide images into large patches and apply computationally expensive linear projections with limited spatial inductive bias, our design is inspired by the efficient spatial decomposition in MobileNetV4 [35]. As shown in Algorithm 1 (lines 1-3), the Stem employs depthwise separable convolutions tailored for ET-flows, capturing localized flow patterns early in the network while significantly reducing operations compared to the standard convolutions in [8]. To the best of our knowledge, such convolutional tokenization, based on depthwise and pointwise convolutions, has not been explored in encrypted traffic detection frameworks.

Finally, positional encoding is applied to preserve spatial relationships, and resulting n-dimensional position embeddings are passed to the wf-Attention modules (line 4).

2) Hierarchical Windowed Factorized Attention: To capture multi-scale flow patterns, we adopt a lightweight hierarchical design that models fine-grained encrypted traffic behaviors at the byte, packet, and flow levels. Each stage consists of a lightweight transformer block composed of Layer Normalization followed by a WF-Attention module. To reduce the quadratic cost of multi-head self-attention, we adopt a low-rank factorized attention mechanism that directly projects token sequences into a shared, low-dimensional space ($r \ll d$), without relying on additional compression matrices, as in Linformer [36], which significantly lowers computational overhead. Furthermore, to avoid costly global attention computations, we employ windowed attention [37] at byte and packet levels, restricting attention to local windows and thus significantly reducing computational overhead while maintaining fine-grained flow representations. This effectively reduces the number of attention computations from $\mathcal{O}(N^2)$ to $\mathcal{O}(N \cdot w)$, where w is the window size.

Although windowed attention limits local interactions at early stages long-range dependencies are preserved through hierarchical composition. Byte- and packet-level encoders induce sparse local attention, while the final flow-level global factorized attention produces dense interactions across all packet tokens and the CLS token. Consequently, the stacked attention layers maintain a fully connected token interaction graph, preserving global contextual modeling with reduced complexity. The attention is computed as [36]:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V,$$

Algorithm 1 LitCViT Pretraining

Input: ET-Flow image $x \in \mathbb{R}^{B\times C\times H\times W}$, dim D, temp. τ
Output: Latent embedding $\mathbf{z}_i$ from encoder $\mathcal{E}$

// Convolutional Patchify Stem
1 $x_1 \leftarrow \text{GELU}(\text{BN}(\text{Conv1D}_{\text{depthwise}}(x)))$
2 $x_2 \leftarrow \text{GELU}(\text{BN}(\text{Conv1D}_{\text{pointwise}}(x_1)))$
3 $\mathbf{E} \leftarrow \text{Flatten}(x_2)$
4 $\mathbf{E} \leftarrow \text{PositionEncoding}(\mathbf{E})$
// Byte-Level Windowed Factorized Attention
5 **for** *each* $\mathbf{E}_i$ *in* $\mathbf{E}$ **do**
6 $\quad \mathbf{W}_i \leftarrow \text{WindowSlice}(\mathbf{E}_i, p, w)$ // Slice into $p \times w$ windows
7 $\quad \mathbf{W}_i \leftarrow \text{LayerNorm}(\mathbf{W}_i)$
8 $\quad \mathbf{A}_i \leftarrow \text{FactorizedAttn}(\mathbf{W}_i; \tau_{\text{attn}})$
9 $\quad \mathbf{P}_i \leftarrow \text{Pool}(\mathbf{A}_i)$ // Packet token
// Inter-Packet Windowed Factorized Attention
10 $\mathbf{P} \leftarrow \text{Stack}([\mathbf{P}_1, \ldots, \mathbf{P}_N])$
11 $\mathbf{P} \leftarrow \text{LayerNorm}(\mathbf{P})$
12 $\mathbf{F}_i \leftarrow \text{FactorizedAttn}(\mathbf{P}; \text{stride}, \tau_{\text{attn}})$
// Intra-Flow global Factorized Attention
13 $\mathbf{F} \leftarrow \text{LayerNorm}(\mathbf{F})$
14 $\mathbf{G}_i \leftarrow \text{FactorizedAttn}(\mathbf{F}, \tau_{\text{attn}})$
// Projection Head
15 $\bar{\mathbf{z}}_i \leftarrow \text{Mean}(\mathbf{G})$ // Global average pooling
16 $\mathbf{z}_i \leftarrow \text{MLP}(\bar{\mathbf{z}}_i)$
// Contrastive Learning
17 **for** *each mini-batch* $\{(\mathbf{z}_i, \mathbf{z}_i^+)\}_{i=1}^N$ **do**
18 $\quad \mathcal{L}_c \leftarrow \text{ntXentLoss}(\{\mathbf{z}_i, \mathbf{z}_i^+\}; \tau)$ // Eq. (1)
19 $\quad$ Backpropagate $\nabla\mathcal{L}_c$ and update $\mathcal{E}$

where W_Q, W_K, $W_V \in \mathbb{R}^{d\times r}$.

$$\text{Attention}(X) = \text{softmax}\left(\frac{QK^\top}{\tau_{attn}}\right)V,$$

where τ_{attn} is a learnable temperature parameter that controls attention sharpness.

The byte-level encoder (lines 5-9) performs factorized self-attention within fixed-size windows of size w over the byte-level representations to capture local dependencies. The resulting features are pooled into packet-level tokens and passed to a second encoder inter-packet wf-Attention (lines 10-12), which performs inter-packet windowed factorized attention to model to capture cross-packet relationships. The Intra-flow global Fact-Attention (lines 13-14) applies global factorized attention across the entire flow to model long-range dependencies. Between each stage, lightweight depthwise convolutions inject spatial inductive biases and compress intermediate features. Finally, the encoded tokens are aggregated via average pooling and passed through a two-layer MLP projection head to produce compact latent embeddings for contrastive learning.

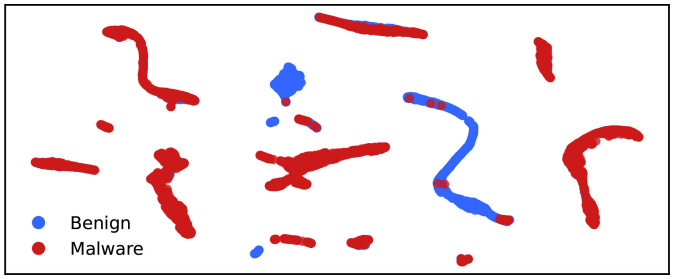

Fig. 2. UMAP visualization of embeddings generated by pre-trained LitCViT.

Upon completion of the pre-training phase, during which the ntXent loss is employed to optimize the model parameters, the encoder is preserved and its pre-trained weights are transferred for use in the fine-tuning phase (Block C).

3) Contrastive Embedding Head: We train LitCVit using the Normalized Temperature-scaled Cross Entropy (ntXent) loss [38] (lines 17-19):

$$\mathcal{L}_{\text{ntXent}} = -\frac{1}{N}\sum_{i=1}^{N}\log\frac{\exp\left(\frac{\mathbf{z}_i\cdot\mathbf{z}_i^+}{\tau}\right)}{\exp\left(\frac{\mathbf{z}_i\cdot\mathbf{z}_i^+}{\tau}\right)+\sum_{j=1,j\neq i}^{N}\exp\left(\frac{\mathbf{z}_i\cdot\mathbf{z}_j}{\tau}\right)}, \quad (1)$$

where $\mathbf{z}_i$ is the embedding of the i-th sample, $\mathbf{z}_i^+$ is a positive (similar) pair, and $\mathbf{z}_j$ are negative (dissimilar) embeddings. τ is a temperature parameter that adjusts the smoothness of the similarity distribution during contrastive learning.

Fig. 2 shows the quality of the learned embeddings at temperature parameter $\tau = 0.3$, visualized using Uniform Manifold Approximation and Projection (UMAP). UMAP projects high-dimensional representations into 2D space while preserving local and global structures, making it suitable for qualitatively assessing clustering behavior. The visualization indicates that LitCVit forms distinct clusters of benign and malicious encrypted traffic flows, highlighting its ability to capture meaningful flow correlations in a self-supervised manner. Quantitatively, the learned embeddings achieve a cluster purity of 0.7579 and a Normalized Mutual Information (NMI) score of 0.508, indicating reasonably good separability of flow behaviors despite encryption. These representations are expected to become more discriminative during supervised fine-tuning. The impact of different τ values on clustering quality is presented in Section VIII. After the pre-training, we freeze the encoder and preserve the learned weights for downstream fine-tuning tasks.

C. Fine-Tuning & Evaluation Phase: Supervised Adaptation for Malicious Traffic Detection

We fine-tune the pre-trained model to detect various types of malicious encrypted traffic, as illustrated in Block C of Fig. 1. Labeled ET-flow images are first processed through the convolutional patchify stem to generate patch embeddings, which are passed through the pre-trained encoder connected to a lightweight MLP-based classifier. The classifier consists of three fully connected layers with GELU activations and a dropout rate of 0.2, ending in a softmax layer. Final predictions are obtained directly from the supervised softmax probability distribution; LitCVit does not employ reconstruction-based

Algorithm 2 Fine-Tuning

Input: Pretrained encoder $\mathcal{E}$, number of classes C, dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$
Output: Fine-tuned classifier $\mathcal{M}$

// Build classifier head
1 $\mathcal{H} \leftarrow$ ClassifierHead()
// Attach classifier to pretrained backbone
2 $\mathcal{M} \leftarrow \mathcal{E} \oplus \mathcal{H}$
3 Freeze($\mathcal{E}$) // Freeze encoder parameters
4 **for** *epoch* $t = 0$ *to* T **do**
5 **if** $t = T_{freeze}$ **then**
6 Unfreeze last two transformer blocks of $\mathcal{E}$
7 **for** *each* (x_i, y_i) *from* $\mathcal{D}$ **do**
8 $\mathbf{z}_i \leftarrow \mathcal{E}(x_i)$ // Generate embeddings
9 $h_1 \leftarrow \text{Dropout}(\text{GELU}(\text{FC}_1(z_i)))$
10 $h_2 \leftarrow \text{Dropout}(\text{GELU}(\text{FC}_2(h_1)))$
11 $\hat{y} \leftarrow \text{softmax}(\text{FC}_{\text{out}}(h_2))$
12 Compute loss $\mathcal{L}_{\text{CE}}$ // Eq. (2)
13 Backpropagate $\nabla\mathcal{L}_{\text{CE}}$ and update parameters

anomaly scoring or threshold-based drift detection. Initially, the encoder is frozen, and only the classification head is trained to prevent overfitting in the early stages. After 100 epochs, the last two encoder blocks are unfrozen to enable the adaptation to the downstream classification task. The model is optimized using the standard multi-class cross-entropy loss $\mathcal{L}_{\text{CE}}$ [39], as defined in Equation 2. This process of fine-tuning is described in Algorithm 2 (lines 3-13).

$$\mathcal{L}_{\text{CE}} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{k=1}^{C} y_{i,k}\log(\hat{y}_{i,k}) \tag{2}$$

Here, N is the number of samples, C is the total number of classes, $y_{\text{i,k}}$ is a binary indicator and $\hat{y}_{\text{i,k}}$ is the predicted probability for class k.

In the final stage of our framework, we evaluate the fine-tuned LitCVit model on labeled test data that was not used during training. This step ensures the model's ability to generalize from the learned representations to classify unseen encrypted traffic flows accurately.

V. Complexity Analysis

We analyze the computational complexity of each component in LitCVit to highlight its architectural efficiency. Despite its multi-stage hierarchical design, the model maintains a low overall computational cost. Similar to [40], we express the complexity in terms of the convolution kernel size k, the number of tokens n, input dimension c_{in}, and output embedding dimensions d. We find both time and space complexities of each component as summarized in Table I.

Convolutional patch embedding stem employs a depthwise and pointwise factorization where the depthwise stage processes channels independently with cost $\mathcal{O}(n \cdot c_{in} \cdot k)$ and the pointwise stage costs $\mathcal{O}(n{\cdot}c_{in}{\cdot}d)$. This yields a total complexity of $\mathcal{O}(n \cdot c_{in}(k + d))$, which is substantially lower than the standard convolutional embedding which costs $\mathcal{O}(n \cdot c_{in} \cdot k \cdot d))$ when ($k \ll d$), while preserving representational capacity. Moreover, hierarchical wf-Attention modules significantly reduce sequence length and the computational cost compared to traditional multi-head attention $\mathcal{O}(n \cdot h \cdot d^2)$ [40], where h is the number of heads and d is the per-head dimension. In our approach, we employ low-rank factorized attention, resulting in a more efficient complexity of $O(n{\cdot}r{\cdot}w)$ operations for local attention within windows of size w and $O(n{\cdot}d{\cdot}r)$ operations for low-rank projections, where $r \ll d$ is the factorization rank.

TABLE I
Asymptotic Complexity of LitCVit Components

Module	Time Complexity	Space Complexity
Conv. Patchify Stem	$\mathcal{O}(n \cdot c_{in}(k + d))$	$\mathcal{O}(C_{in}(k + d)$
wf-Attention modules	$\mathcal{O}(n \cdot r \cdot w + n \cdot d \cdot r)$	$\mathcal{O}(n \cdot (w + d)$
Contras. Embed. Head	$\mathcal{O}(d \cdot h + h \cdot d')$	$\mathcal{O}(h + d')$
ntXent Loss	$\mathcal{O}(B^2 \cdot d')$	$\mathcal{O}(B \cdot d')$
Classifier Head	$\mathcal{O}(d{\cdot}h_1{+}h_1{\cdot}h_2{+}h_2{\cdot}C)$	$\mathcal{O}(h1 + h2 + C)$

n: tokens, C_{in}: input dim., k: kernel size, d: embedding dim., d': projection dim., B: batch size, r: attention rank, w: window size, h: MLP hidden dim.

During pretraining, full forward and backward passes are computed over the entire architecture, including contrastive objectives. The total pre-training complexity is $O\big(n \cdot (r \cdot w + d \cdot r) + d \cdot d' + B^2 \cdot d'\big)$, where the first two terms correspond to the attention and projection layers, and the last term arises from the pairwise similarity computations in ntXent. The time complexity is dominated by the higher order of $\mathcal{O}(B^2 \cdot d')$ due to the quadratic scaling with batch size, while other components scale linearly with the number of tokens n and remain comparatively lightweight. Similarly, space complexity during pre-training is dominated by $\mathcal{O}(B{\cdot}d')$ term from storing projection embeddings for all samples in the batch, with the remaining components contributing only linear growth in n.

In the fine-tuning phase, we compute the total cost by adding the time complexity of convolutional patchify stem, the last two encoder blocks, and the classification head with a softmax cross-entropy loss. Since contrastive learning is only used during pretraining, computational cost is dominated by the two unfrozen encoder blocks, while the classification head contributes a comparatively small cost. The dominant time cost of fine-tuning phase is $\mathcal{O}(n \cdot r \cdot (w + d))$. Space complexity is dominated by storing intermediate activations for the two unfrozen encoder blocks, with the classification head adding a negligible footprint compared to pretraining, where batch-wise projection embeddings dominated memory use. The space complexity of fine-tuning phase is $\mathcal{O}(n{\cdot}(w{+}d))$.

Finally, at inference, the model executes only a lightweight forward pass, without the contrastive head, resulting in an efficient runtime complexity of $\mathcal{O}(n{\cdot}d{\cdot}r)$ and space complexity of $\mathcal{O}(n{\cdot}d)$. As a result, LitCViT achieves a significantly smaller memory footprint and faster inference.

VI. Implementation and Experiments

A. Datasets

To evaluate the performance and generalization of the proposed LitCVit, we utilize four real-world traffic datasets that include benign and malicious network traffic across diverse environments.

USTC-TFC2016 [19] provides encrypted traffic generated by a wide range of benign applications such as Skype, BitTorrent, and YouTube, as well as malicious traffic from ten malware families, including Cridex, Geodo, Htbot, Miuref, Neris, Nsis-ay, Shifu, Tinba, Virut, and Zeus. These malware families represent a wide spectrum of attack behaviors: Tinba and Geodo are banking trojans designed for credential theft, Cridex and Zeus are known for man-in-the-browser attacks. Miuref is associated with click fraud campaigns, while Virut represents a polymorphic file-infector. For our analysis, we retain only complete encrypted traffic flows to ensure data integrity and consistency.

CICIOT-2022 [41] is designed for behavioral analysis and intrusion detection in IoT. It contains both benign and malicious traffic, with two primary attack categories: brute-force attacks (performed using Nmap and Hydra) and flooding attacks.

5GAD-2022 [42] is a 5G network traffic dataset collected by Idaho National Research Laboratory. It captures network traffic in simulated 5G environments to support ML-based attack detection. The dataset encompasses 10 attack categories, including reconnaissance, network reconfiguration, and DoS attacks, providing a comprehensive benchmark for evaluating detection models under diverse and realistic 5G threat scenarios.

Malware Capture Facility Project (MCFP) [43], maintained by the Czech Technical University in Prague, provides a rich repository of network traffic captured in controlled environments. For our study, we extracted malicious samples from diverse malware families, such as BitcoinMiner, Botnet, Cobalt, Dridex, and Trojan Downloader. To create a balanced evaluation set, we additionally selected 600 benign PCAP files from the CTU-Normal series.

IoT-23 [44] is a large-scale IoT network traffic dataset containing benign and malicious traces captured from real IoT devices and botnet infections. It includes labeled attack scenarios alongside normal traffic to support intrusion detection and malware analysis research.

TII-SSRC-23 [45]is a network traffic dataset for IDS research containing benign and malicious traffic with 26 attack classes. It covers 8 main traffic types, including background, multimedia, brute-force, DoS, Mirai botnet, and information-gathering activities.

We utilize USTC-TFC2016 and CIC-IoT-2022 datasets during the pretraining phase, allowing the model to learn generalizable encrypted traffic representations. The rest of datasets are used solely during the fine-tuning phase to evaluate the model's generalization on novel traffic patterns and attack scenarios. Fine-tuning and evaluation are performed on all datasets to ensure a comprehensive assessment of LitCVit's detection performance across diverse network environments.

B. Implementation Overview

We adopt a two-phase training strategy of pre-training followed by fine-tuning, with key hyperparameters summarized in Table II. During pre-training, the ntXent optimization is stabilized via cosine learning rate scheduling with linear warmup. A batch size of 128 provides sufficient in-batch negative diversity for stable contrastive optimization without requiring a memory bank, as supported by our clustering results in Table VIII. The temperature $\tau = 0.3$, achieve the best clustering performance in terms of purity and NMI. All attention modules adopt low-rank factorized self-attention with $r = 16$, balancing expressiveness and efficiency.

TABLE II
KEY HYPERPARAMETERS OF LITCVIT TRAINING

Hyperparameter	Pre-training	Fine-tuning	Selection
Embedding Dim.	128 / 192*	128 / 192	Ablation
Rank (r)	16	16	Ablation
Byte / Packet Wind	5 / 3	5 / 3	Pretraining
Optimizer	AdamW	AdamW	Fixed
LR Schedule	Cosine + Warmup	Cosine + Warmup	Fixed
Weight Decay	10^{-4}	10^{-4}	Fixed
Learning Rate	2×10^{-3}	2×10^{-4}	Tuned
Batch Size	128	64	Tuned
Epochs	—	500	—
Temperature (τ)	0.3	—	Tuned

* See SectionVIII for embedding dimension trade-off analysis.

During fine-tuning, to address severe class imbalance present in datasets such as CICIoT-2022, we employ class-weighted cross-entropy with inverse-frequency weights combined with class-balanced sampling, preventing majority class dominance during both pretraining and fine-tuning. To avoid overfitting during early training, The encoder is initially frozen for the first 100 epochs, after which the last two transformer blocks are unfrozen for deeper fine-tuning. All experiments were conducted on Intel i5-7400 CPU, NVIDIA GeForce GTX 1080 GPU, and 32GB RAM using PyTorch.

Fig. 3 shows the accuracy and loss curves during fine-tuning across all datasets, illustrating LitCVit's stable convergence behavior and robust generalization across diverse traffic scenarios. The model consistently converges within 500 epochs, after which no significant improvements are observed. Training is, therefore, halted to avoid unnecessary computational overhead. Notably, a slight spike in loss and accuracy fluctuation is observed around epoch 100. This is because, at this point, the last two encoder blocks are unfrozen to enable deeper fine-tuning. This controlled unfreezing strategy refines high-level representations while maintaining overall training stability.

C. Baselines and Evaluation Metrics

To evaluate LitCVit, we select baselines spanning feature-driven, end-to-end, and self-supervised pretraining-based techniques. All methods are retrained on the six datasets using their default hyperparameters for fair comparison.

- **Feature-driven:** AppScanner [14] employs flow-level statistical features with Random Forest (RF) and SVM for application traffic classification. Kitsune [15] is an unsupervised method utilizing packet-level metadata features with an autoencoder ensemble to model benign behavior.
- **End-to-end:** 2D-CNN [19] transforms raw encrypted traffic into 2D grayscale images classified via convolutional neural networks.
- **Self-supervised and representation learning:** PEAN [24] pre-trains an attention-based autoencoder to

TABLE III
A PERFORMANCE COMPARISON OF PROPOSED ARCHITECTURE WITH BASE-LINES

	USTCTFC-2016				CICIoT-2022				5GAD-2022			
Method	Acc.	F1	Prec.	Recall	Acc.	F1	Prec.	Recall	Acc.	F1	Prec.	Recall
Appscanner	60.41	62.53	62.23	65.36	76.52	75.69	74.10	63.52	51.03	53.41	53.23	52.13
Kitsune	88.09	82.13	83.64	84.15	87.08	84.26	81.19	78.64	64.23	65.07	62.32	61.23
2D-CNN	92.26	92.05	92.35	92.24	90.07	90.00	90.23	90.41	89.45	87.21	89.23	88.12
PEAN	73.32	67.91	67.25	69.45	61.59	63.63	61.43	61.54	51.12	52.10	52.10	52.45
PERT	90.23	90.12	91.02	91.03	91.09	90.41	91.25	91.36	85.06	85.4	85.69	85.07
ETBERT	87.74	87.47	85.54	85.36	90.35	90.31	89.98	89.78	92.10	93.41	92.07	91.58
YaTC	96.34	96.33	96.36	96.33	95.85	95.12	96.07	95.43	96.04	96.75	95.04	95.36
NetMamba	95.23	95.29	95.56	95.59	92.43	92.49	92.12	92.67	91.34	87.45	87.32	87.19
Pcap-Encoder	91.29	91.52	91.35	91.23	89.45	87.29	87.13	87.35	89.92	89.26	89.72	89.48
LitCVit	**98.37**	**98.32**	**98.25**	**98.37**	**99.12**	**99.11**	**99.14**	**99.02**	**97.24**	**97.25**	**97.26**	**97.24**
	MCFP				**IOT-23**				**TII-SSRC-23**			
Method	Acc.	F1	Prec.	Recall	Acc.	F1	Prec.	Recall	Acc.	F1	Prec.	Recall
Appscanner	64.23	61.03	61.45	63.41	78.08	77.29	77.43	77.12	57.57	57.30	57.24	57.31
Kitsune	78.04	79.12	78.68	78.59	81.83	81.45	80.34	81.11	81.23	91.79	91.32	91.45
2D-CNN	78.1	75.12	78.12	78.12	91.38	91.15	92.25	92.25	78.23	78.05	78.10	77.63
PEAN	78.15	77.18	78.98	78.65	95.69	92.45	91.02	92.45	89.37	90.36	91.27	90.23
PERT	80.22	80.95	81.34	81.03	89.56	88.35	88.33	88.93	92.12	92.45	92.72	92.51
ETBERT	91.36	92.36	91.01	92.55	91.38	89.90	89.14	89.55	83.59	81.12	81.45	81.20
YaTC	94.23	94.15	94.41	94.51	95.78	95.34	95.98	95.31	93.58	91.46	91.50	91.29
NetMamba	81.9	81.39	81.7	89.78	89.53	88.91	99.71	88.93	89.37	87.31	87.54	87.31
PcapEncoder	88.24	88.43	88.51	89.03	91.03	89.20	89.10	91.29	78.24	77.28	77.62	77.29
LitCVit	**97.67**	**97.65**	**97.70**	**97.54**	**98.52**	**98.11**	**98.71**	**98.24**	**98.63**	**98.05**	**98.19**	**98.41**

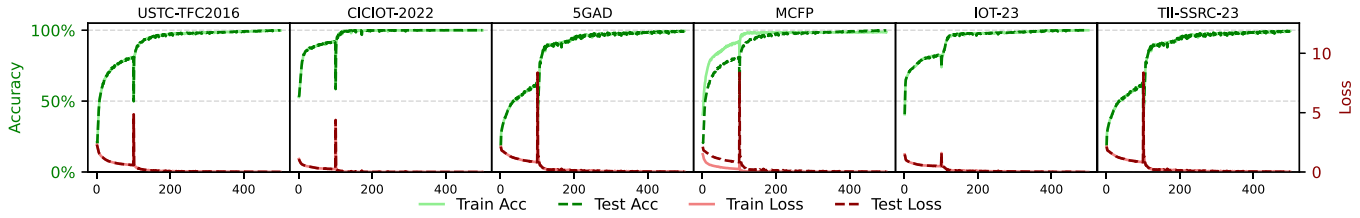

Fig. 3. Accuracy and loss curves during fine-tuning across all datasets. Training converges stably within 500 epochs, with a brief spike around epoch 100 due to unfreezing blocks of the pretrained encoder.

learn generalized traffic embeddings. PERT [46] and ET-BERT [6] adopt BERT-style masked language modeling for contextual traffic representations. YaTC [8] applies a masked autoencoder for pretraining on raw traffic bytes followed by supervised classification. NetMamba [30] is a model based on the Mamba architecture. Pcap-Encoder [9] is a recent representation learning model specifically designed to extract features from protocol headers.

For evaluation, we report accuracy, precision, recall, and F1-score to assess overall classification effectiveness, alongside True Positive Rate (TPR) True Negative Rate (TNR), False Positive Rate (FPR), and False Negative Rate (FNR), to evaluate robustness against false alarms, critical in malicious traffic detection where minimizing missed attacks and false positives is essential.

VII. RESULTS AND DISCUSSION

We evaluate LitCVit across six real-world network traffic datasets, comparing detection effectiveness and computational efficiency against state-of-the-art methods to demonstrate its suitability for practical deployment. Unless otherwise stated, all reported results correspond to an embedding dimension of d = 192, which achieves the best classification performance as determined by the ablation study in Section VIII. For latency-sensitive deployments, d = 128 is recommended as discussed in Section VIII.

A. Detection Performance

Fig. 4 shows confusion matrices illustrating LitCVit's classification performance across all datasets, highlighting its strong ability to distinguish attack types and benign traffic with high confidence and minimal misclassifications. For USTC-TFC2016, LitCVit achieves detection rates exceeding 97% for all malware families, except Geodo and Miuref, which attain detection accuracies of 92% and 96%, respectively. For CICIOT-2022, the model effectively identifies brute-force and flooding attacks while maintaining a low false positive rate for benign flows. Notably, 5GAD-2022 and MCFP, IOT-23, and TII-SSRC-23 were kept completely unseen during pretraining, yet LitCVit shows excellent generalization, achieving above 95% detection rate across all classes. In 5GAD-2022, AMF-FakeInsert attack reaches a detection rate of 95%, while all

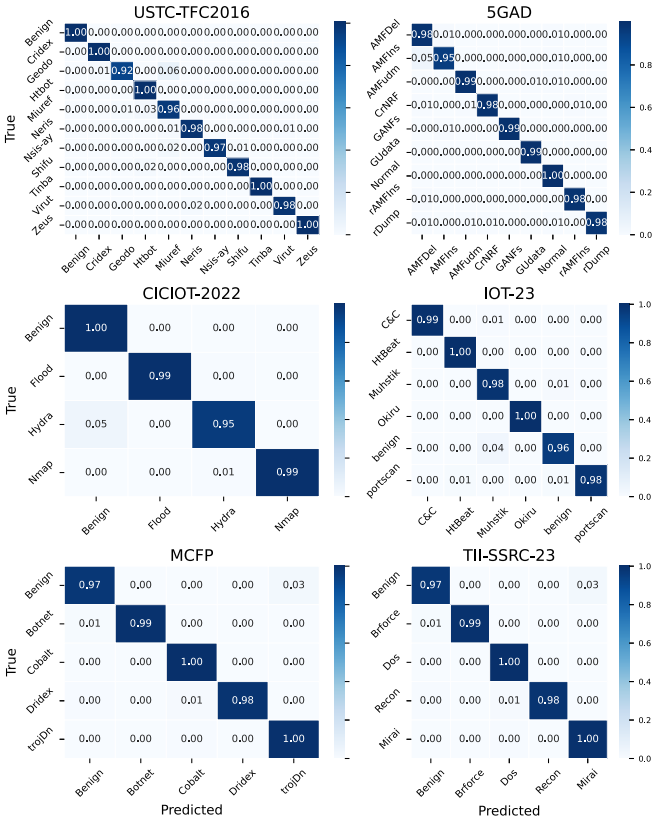

Fig. 4. Performance confusion matrices of all datasets.

other classes surpass 98%. Similarly, on MCFP and IOT-23, all malware classes are detected with at least 97%, despite the diversity and complexity of their behaviors. Similarly, on TII-SSRC-23, LitCVit accurately separates benign and malicious traffic with detection rates exceeding 97%, confirming its robustness across diverse real-world encrypted traffic scenarios.

Fig. 5 demonstrates Precision-Recall (PR) curves to evaluate the proposed framework's performance on datasets with imbalanced class distributions. Minority malicious traffic classes, such as Miuref and Nsis-ay in USTCTFC, and Hydra in CICIOT, pose detection challenges due to their limited representation in the dataset. Despite this, LitCVit maintains robust detection performance. Despite this, LitCVit maintains robust detection performance, achieving PR-AUC values exceeding 0.97 for all malicious traffic classes of USTCTFC, including Tinba, Geodo, Cridex, Zeus, and Virut. In CICIOT, LitCVit attains a reasonable PR-AUC of 0.70 for the minority Hydra class, while achieving perfect PR-AUC scores of 1.00 for Benign, Flood, and Nmap traffic. The reason is that Hydra has a significantly smaller number of samples. On 5GAD, which consists of eight attacks with a nearly balanced distribution, LitCVit consistently achieves high PR-AUC values, ranging from 0.99 to 1.00, across all classes. Similarly, on MCFP, IOT-23, and TII-SSRC-23, LitCVit maintains strong detection performance, achieving PR-AUC scores exceeding 0.99 across all classes. These results highlight the LitCVit's strong ability to maintain high precision and recall across all datasets, indicating robust performance with minimal false predictions.

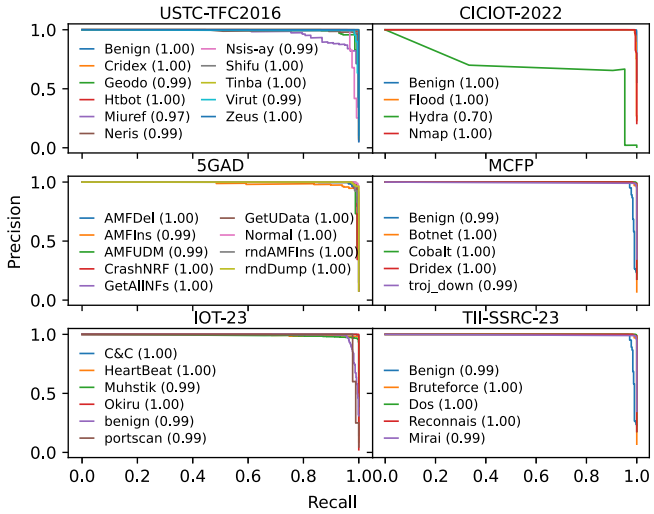

Fig. 5. PR-curves of the proposed LitCVit model on all datasets.

B. Comparative Performance Analysis

Table III presents a comprehensive comparison of LitCVit against state-of-the-art encrypted traffic detection methods. Across six datasets, LitCVit consistently achieves superior detection performance, outperforming all baselines in terms of accuracy, precision, recall, and F1-score. Notably, it surpasses pretraining-based models PEAN and ET-BERT across all metrics, with accuracy gains of 34.17% and 12.15% on USTC-TFC2016, and 60.9% and 9.7% on CICIOT-2022, respectively. Despite being recent strong baselines, NetMamba and PcapEncoder lag behind LitCVit by 7-9% on most datasets. PcapEncoder is designed for packet-level classification using only header information, when evaluated at the flow level, its performance degrades significantly, highlighting the advantage of LitCVit's flow-image representation over header-only packet-level approaches. On unseen datasets, 5GAD-2022, MCFP, IOT-23 and TII-SSRC-23 LitCVit maintains an average accuracy improvement of 5-8% over the best baseline, demonstrating strong generalization capabilities. It also achieves the best precision and recall balance, indicating robustness to diverse encrypted traffic patterns and minimizing false predictions.

Fig. 6 provides a detailed comparison of TPR and FPR across all baselines and datasets, providing a comprehensive evaluation of detection capability and false alarm reduction. LitCVit consistently achieves the highest detection rates across all datasets, exceeding 98% on USTC-TFC, CICIoT-2022, MCFP, IOT-23, and TII-SSRC-23, while maintaining strong performance on the diverse 5GAD-2022 dataset with a TPR of 97%. In contrast, feature-driven methods, AppScanner and Kitsune exhibit significantly lower TPRs (52%–64% and 61%–84%, respectively), highlighting their limited capability in handling encrypted traffic scenarios. In terms of false alarms, LitCVit achieves the lowest FPR, remaining below 0.02 across all datasets. Conversely, AppScanner and the self-supervised PEAN suffer from substantially higher FPR values, reaching up to 0.32 and 0.21, respectively. These results demonstrate that LitCVit not only improves detection

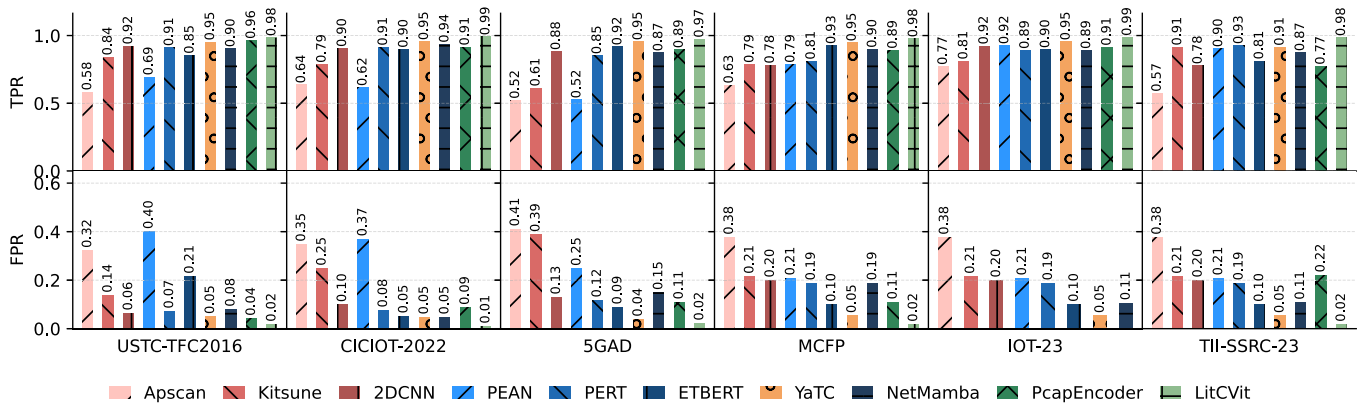

Fig. 6. Comparison of TPR, TNR, FPR, and FNR across all state-of-the-art methods on the USTC-TFC-2016, CICIOT-2022, 5GAD-2022, MCFP, and TII-SSRC-23.

TABLE IV
THE EFFICIENCY COMPARISON OF PROPOSED LITCVIT WITH PRE-TRAINING-BASED TECHNIQUES

Method	GPU Infer (ms)	CPU Infer (ms)	Params. M
PEAN	83.3127	2603.18	2.189
PERT	44.7637	701.23	53.872
ETBERT	46.3854	723.13	132.133
YaTC	20.1452	243.293	1.862
NetMamba	10.1452	112.18	1.91
pcapEncoder	37.771	423	223.50
LitCVit	**2.67**	**5.32**	**0.321**

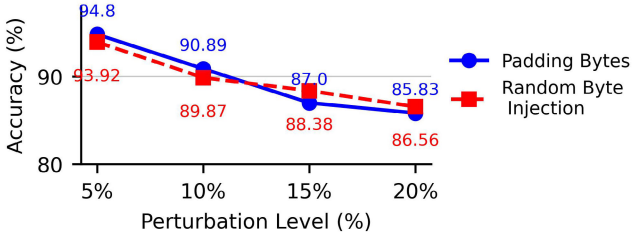

Fig. 7. Robustness of LitCVit under adversarial byte perturbations.

capability but also effectively minimizes false alarms, ensuring reliable encrypted traffic classification.

LitCVit demonstrates robust detection performance with minimal variance across datasets, effectively reducing false alarms and maintaining both high detection accuracy and operational stability in diverse and challenging network environments. Its superior performance is due to its hierarchical attention design, which progressively captures fine-grained byte-level and packet-level patterns and global flow-level dependencies. By employing windowed factorized attention, LitCVit focuses on local contexts while preserving essential token interactions, ensuring robust feature extraction throughout the hierarchy. Additionally, contrastive learning with ntXent loss further enhances the learned embeddings by pulling semantically related ET-flows closer and pushing apart dissimilar ones.

C. Computational and Memory Analysis

Table IV demonstrates that LitCVit offers an efficient solution for encrypted malicious traffic detection., maintaining a minimal memory footprint of approximately 14.83 MB with only 0.321 million parameters. LitCVit achieves a single-sample inference latency of 2.67 ms on GPU, measured with batch size = 1, 50 warm-up iterations, and 300 timed repetitions with GPU synchronization, making it approximately 8.7 times faster than the fastest self-supervised pretraining-based method YaTC while achieving superior detection performance. Moreover, the model requires only 115.55 million multiply-accumulate operations (MMACs), making it highly suitable for deployment in resource-constrained environments.

This efficiency is achieved primarily due to LitCVit's lightweight convolutional patchify stem, which effectively reduces the input dimensionality and the low-rank windowed factorized attention mechanism, which drastically cuts down the number of learnable parameters by performing attention in a compressed token space. These design choices preserve the core capabilities of self-attention while significantly lowering computational overhead.

D. Adversarial Robustness Analysis

To assess robustness against simple adversarial manipulation, two perturbation strategies are simulated: padding-byte insertion and random byte injection. Padding-byte insertion appends null bytes to packet payloads, simulating traffic padding commonly used in protocol obfuscation. Random byte injection replaces a random fraction of payload pixel values with uniform random bytes, simulating arbitrary payload corruption. All perturbations are applied exclusively to payload rows, preserving header structure to reflect realistic encrypted traffic manipulation. Perturbation experiments are conducted on USTC-TFC2016, the most class-diverse dataset with 11 traffic categories. As shown in Fig. 7, LitCVit maintains accuracy above 85% under both strategies even at 20% perturbation, with a maximum degradation of 8.97% for padding-byte insertion and 7.36% for random byte injection. The convergence of both curves at higher perturbation levels indicates consistent resilience independent of perturbation type. These results demonstrate that LitCVit captures structural

TABLE V
COMPONENT-WISE ABLATION STUDY OF THE PROPOSED LITCVIT ACROSS SIX BENCHMARK DATASETS

	USTC-TFC2016		CICIOT-2022		5GAD		MCFP		IOT-23		TII-SSRC-23	
Variant	Acc.	F1	Acc.	F1	Acc.	F1	Acc.	F1	Acc.	F1	Acc.	F1
LitCVit (PT + WA)	**98.37**	**98.32**	**99.12**	**99.11**	98.87	98.23	**97.67**	**97.65**	98.52	98.11	98.63	98.05
Full Rank	97.31	94.90	98.46	98.20	**98.18**	**98.45**	96.98	97.05	**99.21**	**99.12**	**98.14**	**98.45**
w/o Windowed Attn.	96.98	96.02	98.16	97.79	98.18	97.25	96.54	96.21	97.35	97.19	97.43	97.18
w/o Pre-training	95.69	94.78	95.54	95.57	95.14	95.58	91.56	91.04	92.40	92.83	94.36	94.81
w/o Conv. Stem	94.53	94.31	96.41	96.04	93.94	93.89	92.26	92.94	97.38	96.49	94.00	94.29
w/o Packet Encoder	93.34	91.24	91.37	91.80	89.34	89.38	91.24	92.43	92.47	92.35	90.29	90.42
w/o Byte Encoder	88.37	85.20	83.29	83.18	78.20	79.21	84.16	83.95	81.75	81.97	78.31	78.29
w/o Flow Encoder	45.21	30.29	51.24	52.43	35.98	35.29	56.89	56.74	48.53	48.98	34.19	33.63

Note: PT: self-supervised pre-training, WA: windowed factorized attention. Best results per column are **bold**, second best are underlined.

and semantic traffic patterns at multiple granularities rather than relying on precise byte positions, making it inherently resilient to simple byte-level adversarial perturbations.

E. Resource-Constrained Inference Analysis

To evaluate deployment feasibility on resource-constrained platforms, LitCVit is evaluated under GPU and CPU settings. Specifically, single-thread CPU inference is conducted on a system with 8 GB RAM and an Intel i5-7400 @ 3.00 GHz, deliberately chosen to simulate resource-constrained deployment conditions. With dim = 128, rank = 16, the model contains only 0.32 M parameters and requires 115.55 MMac. It achieves 2.67 ms per-sample latency on GPU, 5.32 ms under multi-thread CPU execution, and 8.70 ms $\pm$ 0.98 in single-thread mode. The model maintains a compact memory footprint (14.83 MB total, nearly 4.66 MB runtime overhead), demonstrating real-time capability and suitability for resource-constrained platforms. Physical validation on embedded platforms such as Raspberry Pi or NVIDIA Jetson is left as future work.

VIII. ABLATION STUDY

A. Component-Wise Ablation Study

Table V presents a component-wise ablation study of the proposed LitCVit on six benchmark datasets. Removing the flow-level encoder causes the most severe degradation, with Macro-F1 dropping to as low as 30.29% on USTC-TFC2016 and 33.63% on TII-SSRC-23, confirming the importance of global context aggregation across packet tokens for reliable traffic classification. Removing the byte-level encoder results in the second largest performance drop, highlighting the importance of fine-grained byte-level feature extraction in capturing low-level traffic patterns. Without self-supervised pre-training, the performance declines consistently across datasets, demonstrating that pre-training provides meaningful semantic representations. Overall, each component contributes meaningfully to performance, with the hierarchical encoder structure and self-supervised pre-training providing the most significant gains.

B. Impact of Rank

We conduct an ablation study over rank $r \in \{8, 16, 32, 64\}$ and embedding dimension $d \in \{128, 156, 192\}$ to analyze their effect on model complexity and inference efficiency. As shown in Fig. 8, parameters and MMACs increase monotonically with both r and d, confirming predictable scaling behavior. GPU latency remains relatively stable across ranks, while CPU latency grows more noticeably with dimension, reflecting limited CPU parallelism for larger matrix operations. Notably, rank $r = 16$ achieves the lowest inference latency across all dimensions. Detection accuracy remains stable across ranks, and the marginal gains at higher ranks confirm that encrypted traffic features lie in a low-dimensional subspace where compact factorized attention is sufficient. Thus, $r = 16$ and $d = 128$ are adopted as default for the optimal accuracy–efficiency trade-off.

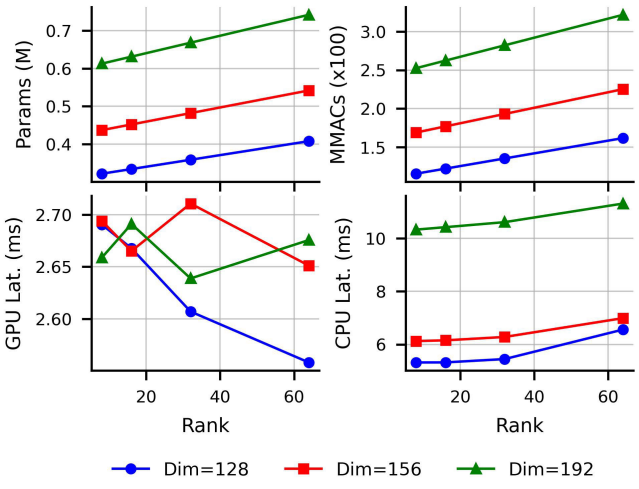

Fig. 8. Model complexity and inference latency vs. rank $\in \{8, 16, 32, 64\}$ for embedding dimensions $\in \{128, 156, and 192\}$. x-axis: Rank in all subplots.

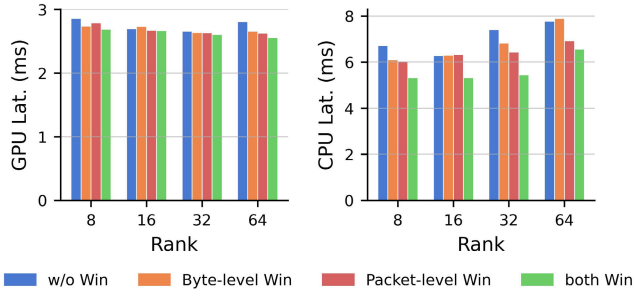

Fig. 9. Inference latency comparison of windowed attention configurations at dim = 128.

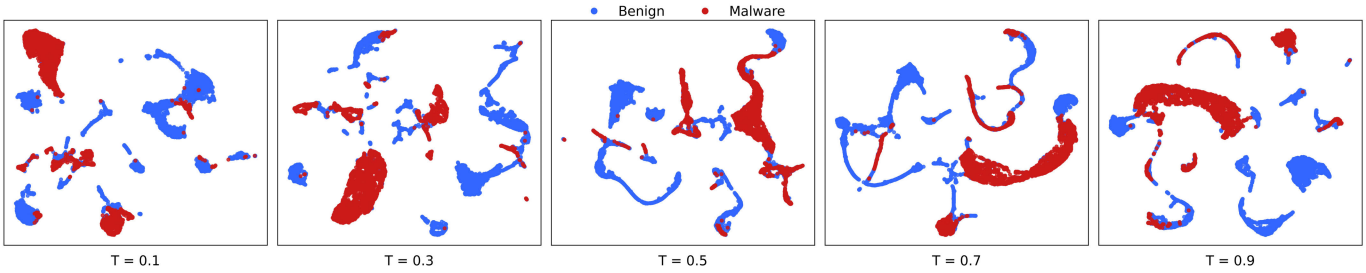

Fig. 10. UMAP embeddings of encrypted benign and malicious traffic under different temperature values τ of ntXent loss.

TABLE VI

ABLATION STUDY ON VARIOUS WINDOW CONFIGURATIONS. PERFORMANCE IS BEST WITH BOTH PACKET AND BYTE-LEVEL WINDOWS

Window Config.	USTC-TFC2016		CICIOT-2022		MCFP	
	Acc	F1	Acc	F1	Acc	F1
w/o Windows	97.58	97.02	98.16	97.79	96.54	96.21
Byte-level Window	95.80	93.89	95.23	93.56	94.23	94.23
Packet-level Window	95.08	92.00	93.56	93.54	94.23	94.72
with both Windows	**98.37**	**98.32**	**99.12**	**99.11**	**97.67**	**97.65**

TABLE VII

PERFORMANCE AND EFFICIENCY COMPARISON OF CONVENTIONAL LIGHTWEIGHT MODELS WITH LITCVIT

Model	Macro-F1 (%)			Efficiency		
	USTC	CICIOT	MCFP	cpu Lat (ms)	Param.(K)	MACs(M)
RF	84.72	84.03	62.05	27.46	–	–
XGB	92.70	91.39	63.54	1.75	–	–
2DCNN	89.97	83.67	64.54	0.77	95.7	15.75
BiLSTM	65.04	71.52	70.71	0.61	179.2	7.07
GRU	70.66	71.41	65.71	6.99	135.7	5.30
LitCVit	**98.37**	**99.11**	**97.65**	**5.57**	**321.81**	**115.55**

TABLE VIII

EFFECT OF TEMPERATURE PARAMETER τ ON CONTRASTIVE VIT

τ	Loss	Cluster Purity	NMI	ARI	Silhouette Score
0.1	**0.758**	0.7198	0.464	0.2166	0.4077
0.3	2.335	**0.7579**	**0.508**	0.2556	0.4213
0.5	3.169	0.7349	0.476	0.2333	0.6069
0.7	3.611	0.7352	0.485	0.2774	0.7524
0.9	3.871	0.7228	0.5	**0.3743**	**0.8476**

C. Effect of Windowed Attention

Ablation results in Fig. 9 show that GPU inference latency remains invariant across all window configurations (~ 2.6–2.9 ms), indicating that windowed attention introduces negligible computational overhead on GPU. On CPU, enabling both byte and packet windows reduces latency by 15 – 25% compared to global attention at dim = 128. This suggests that restricting the attention span reduces redundant computation at the CPU level, where parallelism is more limited than on GPU. Table VI further reports consistent improvements in Accuracy and Macro-F1 when both windows are applied. We attribute this to the hierarchical design, where early local attention reduces premature global token mixing and encourages spatially coherent byte and packet patterns, while long-range dependencies are preserved by the flow-level transformer through full global attention across packet tokens.

D. Performance Comparison With Conventional Models

To justify our architecture, we compare LitCVit against tree-based (RF, XGBoost) and lightweight neural baselines (2DCNN, GRU, BiLSTM) under identical settings. Since tree-based and lightweight neural models are inherently CPU-based, CPU latency is used as the common efficiency metric for fair comparison. Tree-based models use flattened 1600-d ET-flow vectors, while neural baselines use flow-image representations. As shown in Table VII, XGBoost exceeds 90% Macro F1 on USTCTFC and CICIOT but degrades on MCFP, indicating limited generalization. RF incurs high latency (27.46 ms) despite zero trainable parameters due to sequential tree traversal. Lightweight neural models offer low latency but inconsistent performance (GRU/BiLSTM up to 71.52% Macro F1) with notable MCFP degradation. LitCVit consistently achieves the highest Macro F1 across all datasets with a compact footprint and competitive latency, demonstrating superior generalization-efficiency balance.

E. Impact of Temperature Parameter

The effect of Temperature parameter τ in ntXent loss on clustering quality is illustrated in Fig. 10 and Table VIII. Lower values of τ (e.g., 0.1) yield reduced loss; however, higher values (e.g., 0.9) resulted in improved global structure, as reflected by higher NMI, Adjusted Rand Index (ARI), and Silhouette scores, indicating better separation between benign and malicious traffic. The best overall clustering performance is achieved at τ = 0.3, which provides an effective trade-off between local compactness and global separability in the learned representations.

IX. CONCLUSION

In this paper, we propose a lightweight self-supervised contrastive vision transformer framework for encrypted malicious traffic detection. The model integrates an efficient convolutional patchify stem to encode local spatial features and employs low-rank windowed factorized self-attention mechanisms at byte, packet, and flow levels to multi-granularity representations. Additionally, a contrastive pretraining objective enables the model to learn ET-flow semantic features without relying on labeled data. Extensive experiments on six real-world datasets demonstrate that the proposed framework outperforms the state-of-the-art method, including recent baselines, achieving 5-8% average accuracy improvement on unseen datasets while maintaining a compact footprint of 0.321 M parameters and 2.67 ms GPU inference latency approximately 8.7 times faster than YaTC. These results

validate LitCVit's strong generalization across diverse attack scenarios while maintaining high efficiency and low inference time, making it highly suitable for deployment in resource-constrained environments. Future work will explore model compression via pruning and quantization, as well as physical validation on embedded platforms such as Raspberry Pi and NVIDIA Jetson.

REFERENCES

[1] M. A. I. Mallick and R. Nath, "Navigating the cyber security landscape: A comprehensive review of cyber-attacks, emerging trends, and recent developments," *World Sci. News*, vol. 190, no. 1, pp. 1–69, 2024.

[2] J. Ahn, R. Hussain, K. Kang, and J. Son, "Exploring encryption algorithms and network protocols: A comprehensive survey of threats and vulnerabilities," *IEEE Commun. Surveys Tuts.*, vol. 27, no. 6, pp. 3587–3614, Dec. 2025.

[3] M. Shen et al., "Machine learning-powered encrypted network traffic analysis: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 1, pp. 791–824, 1st Quart., 2023.

[4] M. Almehdhar et al., "Deep learning in the fast lane: A survey on advanced intrusion detection systems for intelligent vehicle networks," *IEEE Open J. Veh. Technol.*, vol. 5, pp. 869–906, 2024.

[5] Y. Feng et al., "Unmasking the internet: A survey of fine-grained network traffic analysis," *IEEE Commun. Surveys Tuts.*, vol. 27, no. 6, pp. 3672–3709, Dec. 2025.

[6] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proc. ACM Web Conf.*, Apr. 2022, pp. 633–642.

[7] S. Cui, C. Dong, M. Shen, Y. Liu, B. Jiang, and Z. Lu, "CBSeq: A channel-level behavior sequence for encrypted malware traffic detection," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 5011–5025, 2023.

[8] R. Zhao et al., "A novel self-supervised framework based on masked autoencoder for traffic classification," *IEEE/ACM Trans. Netw.*, vol. 32, no. 3, pp. 2012–2025, Jun. 2024.

[9] Y. Zhao, G. Dettori, M. Boffa, L. Vassio, and M. Mellia, "The sweet danger of sugar: Debunking representation learning for encrypted traffic classification," in *Proc. ACM SIGCOMM Conf.*, Sep. 2025, pp. 296–310.

[10] L. Peng, X. Xie, S. Huang, Z. Wang, and Y. Cui, "Ptu: Pre-trained model for network traffic understanding," in *Proc. IEEE 32nd Int. Conf. Netw. Protocols (ICNP)*, Oct. 2024, pp. 1–12.

[11] G. Zhou, X. Guo, Z. Liu, T. Li, Q. Li, and K. Xu, "TrafficFormer: An efficient pre-trained model for traffic data," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2025, pp. 1844–1860.

[12] S. Naveen and M. R. Kounte, "Compact optimized deep learning model for edge: A review," *Int. J. Electr. Comput. Eng. (IJECE)*, vol. 13, no. 6, p. 6904, Dec. 2023.

[13] A. A. Bhutta, M. U. Nisa, and A. N. Mian, "Lightweight real-time WiFi-based intrusion detection system using LightGBM," *Wireless Netw.*, vol. 30, no. 2, pp. 749–761, Feb. 2024.

[14] V. F. Taylor, R. Spolaor, M. Conti, and I. Martinovic, "Robust smartphone app identification via encrypted network traffic analysis," *IEEE Trans. Inf. Forensics Security*, vol. 13, no. 1, pp. 63–78, Jan. 2018.

[15] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, San Diego, CA, USA, 2018.

[16] C. Fu, Q. Li, M. Shen, and K. Xu, "Frequency domain feature based robust malicious traffic detection," *IEEE/ACM Trans. Netw.*, vol. 31, no. 1, pp. 452–467, Feb. 2023.

[17] B. Anderson, S. Paul, and D. McGrew, "Deciphering malware's use of TLS (without decryption)," *J. Comput. Virol. Hacking Techn.*, vol. 14, no. 3, pp. 195–211, Aug. 2018.

[18] C. Kondaiah, A. R. Pais, and R. S. Rao, "Enhanced malicious traffic detection in encrypted communication using TLS features and a multi-class classifier ensemble," *J. Netw. Syst. Manage.*, vol. 32, no. 4, p. 76, Oct. 2024.

[19] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, Jan. 2017, pp. 712–717.

[20] L. Yu et al., "PBCNN: Packet bytes-based convolutional neural network for network intrusion detection," *Comput. Netw.*, vol. 194, Jul. 2021, Art. no. 108117.

[21] T. Shapira and Y. Shavitt, "FlowPic: A generic representation for encrypted traffic classification and applications identification," *IEEE Trans. Netw. Service Manage.*, vol. 18, no. 2, pp. 1218–1232, Jun. 2021.

[22] A. F. Diallo and P. Patras, "Cluster and conquer: Malicious traffic classification at the edge," *IEEE Trans. Netw. Service Manage.*, vol. 21, no. 3, pp. 2700–2714, Jun. 2024.

[23] S. Guthula, R. Beltiukov, N. Battula, W. Guo, A. Gupta, and I. Monga, "NetFound: Foundation model for network security," 2023, *arXiv:2310.17025*.

[24] P. Lin, K. Ye, Y. Hu, Y. Lin, and C.-Z. Xu, "A novel multimodal deep learning framework for encrypted traffic classification," *IEEE/ACM Trans. Netw.*, vol. 31, no. 3, pp. 1369–1384, Jun. 2023.

[25] H. Y. He, Z. Guo Yang, and X. N. Chen, "PERT: Payload encoding representation from transformer for encrypted traffic classification," in *Proc. ITU Kaleidoscope, Ind.-Driven Digit. Transformation (ITU K)*, Dec. 2020, pp. 1–8.

[26] X. Zang, T. Wang, X. Zhang, J. Gong, P. Gao, and G. Zhang, "Encrypted malicious traffic detection based on natural language processing and deep learning," *Comput. Netw.*, vol. 250, Aug. 2024, Art. no. 110598.

[27] J. Qu, X. Ma, and J. Li, "TrafficGPT: Breaking the token barrier for efficient long traffic analysis and generation," 2024, *arXiv:2403.05822*.

[28] X.-Y. Chen, L. Han, D.-C. Zhan, and H.-J. Ye, "Miett: Multi-instance encrypted traffic transformer for encrypted traffic classification," in *Proc. 39th AAAI Conf. Artif. Intell.*, vol. 39, 2025, pp. 15922–15929.

[29] Z. Hang, Y. Lu, Y. Wang, and Y. Xie, "Flow-MAE: Leveraging masked AutoEncoder for accurate, efficient and robust malicious traffic classification," in *Proc. 26th Int. Symp. Res. Attacks, Intrusions Defenses*, Oct. 2023, pp. 297–314.

[30] T. Wang, X. Xie, W. Wang, C. Wang, Y. Zhao, and Y. Cui, "Netmamba: Efficient network traffic classification via pre-training unidirectional mamba," in *Proc. IEEE 32nd Int. Conf. Netw. Protocols (ICNP)*, Oct. 2024, pp. 1–11.

[31] M. Zhan, J. Yang, D. Jia, and G. Fu, "EAPT: An encrypted traffic classification model via adversarial pre-trained transformers," *Comput. Netw.*, vol. 257, Feb. 2025, Art. no. 110973.

[32] C. Fu, Q. Li, E. Bertino, and K. Xu, "Training with only 1.0% samples: Malicious traffic detection via cross-modality feature fusion," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2025, pp. 3930–3944.

[33] N. Wickramasinghe, A. Shaghaghi, G. Tsudik, and S. Jha, "SoK: Decoding the enigma of encrypted network traffic classifiers," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2025, pp. 1825–1843.

[34] A. Dosovitskiy et al., "An image is worth 16×16 words: Transformers for image recognition at scale," 2020, *arXiv:2010.11929*.

[35] D. Qin et al., "Mobilenetv4: Universal models for the mobile ecosystem," in *Proc. Eur. Conf. Comput. Vis.*, 2024, pp. 78–96.

[36] S. Wang, B. Z. Li, M. Khabsa, H. Fang, and H. Ma, "Linformer: Self-attention with linear complexity," 2020, *arXiv:2006.04768*.

[37] Z. Liu et al., "Swin transformer: Hierarchical vision transformer using shifted windows," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 10012–10022.

[38] C. Wen, X. Li, H. Huang, Y.-S. Liu, and Y. Fang, "3D shape contrastive representation learning with adversarial examples," *IEEE Trans. Multimedia*, vol. 27, pp. 679–692, 2023.

[39] Z. Zhang and M. R. Sabuncu, "Generalized cross entropy loss for training deep neural networks with noisy labels," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, 2018, pp. 8792–8802.

[40] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 5998–6008.

[41] S. Dadkhah, H. Mahdikhani, P. K. Danso, A. Zohourian, K. A. Truong, and A. A. Ghorbani, "Towards the development of a realistic multidimensional IoT profiling dataset," in *Proc. 19th Annu. Int. Conf. Privacy, Secur. Trust (PST)*, Aug. 2022, pp. 1–11.

[42] C. Coldwell et al., "Machine learning 5G attack detection in programmable logic," in *Proc. IEEE Globecom Workshops (GC Wkshps)*, Dec. 2022, pp. 1365–1370.

[43] *Stratosphere Malware Capture Facility Project Datasets*. Accessed: Jul. 2025. [Online]. Available: https://mcfp.felk.cvut.cz/publicDatasets/

[44] S. Garcia, A. Parmisano, and M. J. Erquiaga, "IoT-23: A labeled dataset with malicious and benign IoT network traffic," Version 1.0.0, Zenodo, Stratosphere Lab., Czech Tech. Univ. Prague, Prague, Czech Republic, 2020, doi: 10.5281/zenodo.4743746.

[45] D. Herzalla, W. T. Lunardi, and M. Andreoni, "TII-SSRC-23 dataset: Typological exploration of diverse traffic patterns for intrusion detection," *IEEE Access*, vol. 11, pp. 118577–118594, 2023.

[46] H. He, Z. Yang, and X. Chen, "Payload encoding representation from transformer for encrypted traffic classification," *ZTE Commun.*, vol. 19, no. 4, p. 90, 2021.